

CS486/686: Introduction to Artificial Intelligence

Lecture 8b - Supervised Machine Learning

Jesse Hoey & Victor Zhong

School of Computer Science, University of Waterloo

March 3, 2025

Readings: Poole & Mackworth Chap. 7.3.2, 7.4-7.5, 8

Linear Regression

Linear regression is a model in which the output is a linear function of the input features

$$\hat{Y}_{\vec{w}}(e) = w_0 + w_1X_1(e) + \cdots + w_nX_n(e)$$

$$\hat{Y}_{\vec{w}}(e) = \sum_{i=0}^n w_iX_i(e)$$

where $\vec{w} = \langle w_0, w_1, w_2, \dots, w_n \rangle$. We invent a new feature $X_0 \equiv 1$, to make it not a special case.

Linear Regression

Linear regression is a model in which the output is a linear function of the input features

$$\hat{Y}_{\vec{w}}(e) = w_0 + w_1 X_1(e) + \cdots + w_n X_n(e)$$

$$\hat{Y}_{\vec{w}}(e) = \sum_{i=0}^n w_i X_i(e)$$

where $\vec{w} = \langle w_0, w_1, w_2, \dots, w_n \rangle$. We invent a new feature $X_0 \equiv 1$, to make it not a special case.

The **sum of squares error** on examples E for output Y is:

$$Error(E, \vec{w}) = \sum_{e \in E} (Y(e) - \hat{Y}_{\vec{w}}(e))^2 = \sum_{e \in E} \left(Y(e) - \sum_{i=0}^n w_i X_i(e) \right)^2$$

Goal: find weights that minimize $Error(E, \vec{w})$

Finding weights that minimize $Error(E, \vec{w})$

Find the minimum **analytically**.

Effective when it can be done (e.g., for linear regression). If

- $\vec{y} = [Y(e_1), Y(e_2), \dots, Y(e_M)]$ is a vector of the output features for the M examples
- X is a matrix where the j^{th} column is the values of the input features for the j^{th} example
- $\vec{w} = [w_0, w_1, \dots, w_n]$ is a vector of the weights

then,

$$\begin{aligned}\vec{y} &= \vec{w}X \\ \vec{y}X^T(XX^T)^{-1} &= \vec{w}\end{aligned}$$

$(XX^T)^{-1}$ is the **pseudo-inverse**, which may not be easily computable

Finding weights that minimize $Error_E(\vec{w})$

Find the minimum **iteratively**.

Works for larger classes of problems (not just linear).

Gradient descent:

$$w_i \leftarrow w_i - \eta \frac{\partial Error(E, \vec{w})}{\partial w_i}$$

η is the gradient descent step size, the **learning rate**.

If

$$Error(E, \vec{w}) = \sum_{e \in E} (Y(e) - \hat{Y}_{\vec{w}}(e))^2 = \sum_{e \in E} \left(Y(e) - \sum_{i=0}^n w_i X_i(e) \right)^2$$

then **update rule:**

$$w_i \leftarrow w_i + \eta \sum_{e \in E} \left(Y(e) - \sum_{i=0}^n w_i X_i(e) \right) X_i(e)$$

where we have set $\eta \rightarrow 2\eta$ (arbitrary scale)

Incremental Gradient Descent for Linear Regression

- 1: LinearLearner(X, Y, E, η)
- 2: **Input:** set of input features, $X = \{X_1, \dots, X_n\}$
- 3: Y : output feature
- 4: E : set of examples to learn from
- 5: η : learning rate
- 6: Initialize weights $\vec{w}$ randomly
- 7: **Repeat**
- 8: **for each** $e \in E$
- 9: $\delta \leftarrow Y(e) - \sum_{i=0}^n w_i X_i(e)$
- 10: **for each** $i \in \{0, \dots, n\}$
- 11: $w_i \leftarrow w_i + \eta \delta X_i(e)$
- 12: **Until** Stopping criterion is true

Stochastic and Batched Gradient Descent

- Algorithm on the last slide is **incremental** gradient descent
- If examples are chosen randomly at line 8 then its **stochastic gradient descent**.
- **Batched gradient descent**:
 - process a batch of size n before updating the weights
 - if n is all the data, then its **gradient descent**
 - if $n = 1$, its **incremental gradient descent**
- Incremental can be more efficient than batch, but convergence not guaranteed
 - weight updates are done immediately
 - but might “undo” the work of other weight updates creating an oscillation

Linear Classifier

- Assume we are doing **binary classification**, with classes $\{0, 1\}$
- There is no point in making a prediction of less than 0 or greater than 1, especially when we use a probabilistic interpretation
- A **squashed linear function** is of the form:

$$\begin{aligned}\hat{Y}_{\vec{w}}(e) &= f(w_0 + w_1 X_1(e) + \cdots + w_n X_n(e)) \\ &= f\left(\sum_{i=0}^n w_i X_i(e)\right)\end{aligned}$$

where f is an **activation function**.

- A simple activation function is the **step function**:

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Gradient Descent for Linear Classifiers

If the activation function is **differentiable**, we can use **gradient descent** to update the weights. The sum of squares error:

$$Error(E, \vec{w}) = \sum_{e \in E} \left(Y(e) - f \left(\sum_{i=0}^n w_i X_i(e) \right) \right)^2$$

The partial derivative with respect to weight w_i for example e is:

$$\frac{\partial Error(e, \vec{w})}{\partial w_i} = -2\delta f' \left(\sum_i w_i X_i(e) \right) X_i(e)$$

where $\delta = (Y(e) - f(\sum_{i=0}^n w_i X_i(e)))$.

Thus, each example e updates each weight w_i by

$$w_i \leftarrow w_i + \eta \delta f' \left(\sum_i w_i X_i(e) \right) X_i(e)$$

The sigmoid or logistic activation function

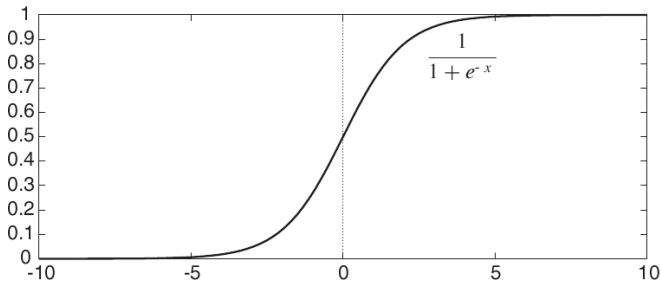


Figure 7.7: The sigmoid or logistic function

$$f(x) = \frac{1}{1 + e^{-x}}$$
$$f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = f(x)(1 - f(x))$$

so $f'(x)$ can be computed from $f(x)$

The sigmoid or logistic activation function

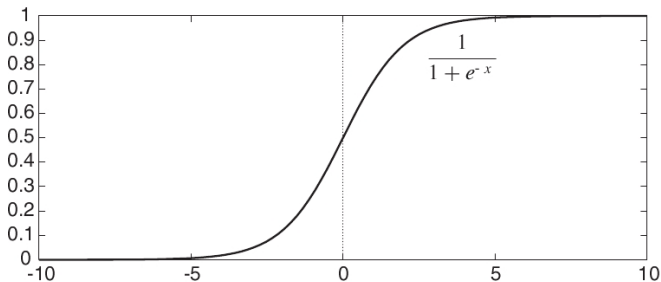


Figure 7.7: The sigmoid or logistic function

$$f(x) = \frac{1}{1 + e^{-x}}$$
$$f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = f(x)(1 - f(x))$$

so $f'(x)$ can be computed from $f(x)$

Discussion Board Example

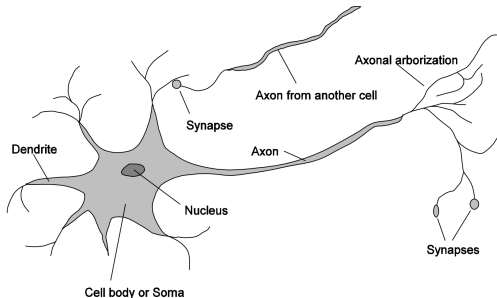


$$\widehat{Reads}(e) = \text{sigmoid}(-8 + 7 * Short(e) + 3 * New(e) + 3 * Known(e))$$

Using the 18 training examples from lecture 4, this can be found in about 3000 iterations with a learning rate of $\eta = 0.05$

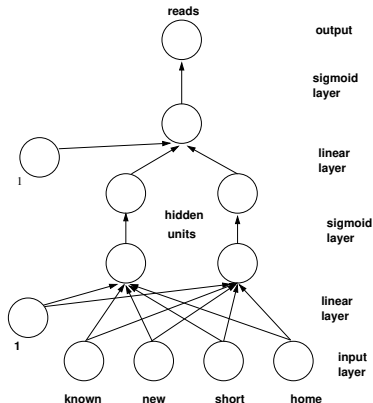
Neural Networks

- inspired by **biological networks** (brain)
- connect up many simple **units**
- simple neuron: **threshold and fire**
- can help **gain understanding** of how biological intelligence works



Neural Networks

- can learn the **same things that a decision tree** can
- imposes **different learning bias** (way of making new predictions)
- **back-propagation** learning: errors made are propagated backwards to change the weights
- often the linear and sigmoid layers are treated as a single layer



Neural Networks Basics

- Each node j has a set of weights $\mathbf{w}_j = [w_{j0}, w_{j1}, \dots, w_{jN}]$
- Each node j receives inputs $\mathbf{v} = [v_0, v_1, \dots, v_N]$, which are either the example features $[x_0, x_1, \dots, x_N]$, or the output of a previous layer ℓ $[z^{(\ell)}_0, z^{(\ell)}_1, \dots, z^{(\ell)}_N]$
- number of weights = number of parents + 1
($v_0 = 1$ constant bias term)
- output (z) is the activation function output

$$z_j = f \left(\sum_i w_{ij} v_i \right)$$

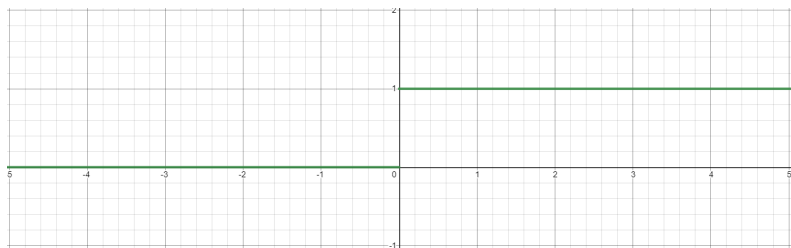
necessarily **non-linear** because

A linear function of a linear function is a ...

linear function

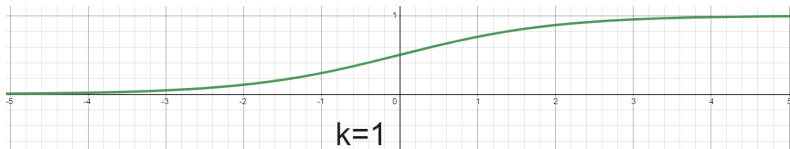
Common Activation Functions

- **Step function:** $f(x) = 1$ if $x > 0$. $f(x) = 0$ if $x \leq 0$.
 - Simple to use, but not differentiable.
 - Not used in practice, but useful to explain concepts.



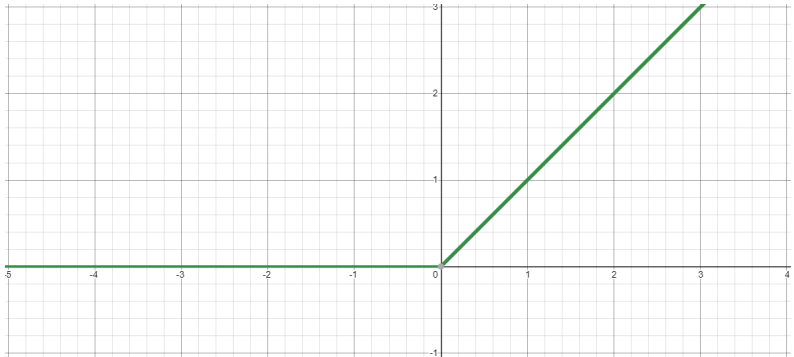
Common Activation Functions (cont.)

- **Sigmoid function:** $f(x) = \frac{1}{1 + e^{-kx}}$.
 - For very large or very small x , $f(x)$ is very close to 1 or 0.
 - Can approximate the step function by tuning k . As k increases, the sigmoid function becomes steeper and is closer to the step function. Usually in practice $k = 1$.
 - Differentiable.
 - **Vanishing gradient problem:** when x is very large or very small, $f(x)$ responds little to changes in x . The network does not learn further or learns very slowly.
 - Computationally expensive.



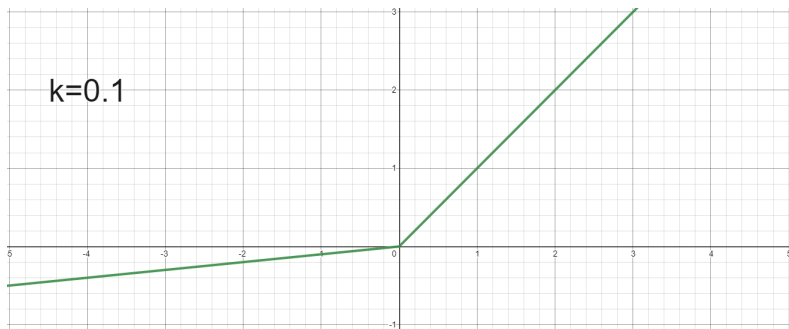
Common Activation Functions (cont.)

- **Rectified linear unit (ReLU):** $f(x) = \max(0, x)$.
 - Computationally efficient; network converges quickly.
 - Differentiable.
 - **The dying ReLU problem:** when inputs approach 0 or are negative, the gradient becomes 0 and the network cannot learn.



Common Activation Functions (cont.)

- **Leaky ReLU:** $f(x) = \max(0, x) + k \cdot \min(0, x)$
 - Small positive slope k in the negative area. Enables learning for negative input values.



Connecting the neurons together into a network

- **Feedforward network**

- Forms a directed acyclic graph (no loops).
- Have connections only in one direction.
- Represents a function of its inputs.

- **Recurrent network**

- Feeds its outputs back into its inputs.
- Can support short-term memory. For the given inputs, the behaviour of the network depends on its initial state, which may depend on previous inputs.
- The model is more interesting, but more difficult to understand and to learn.

Learning Weights

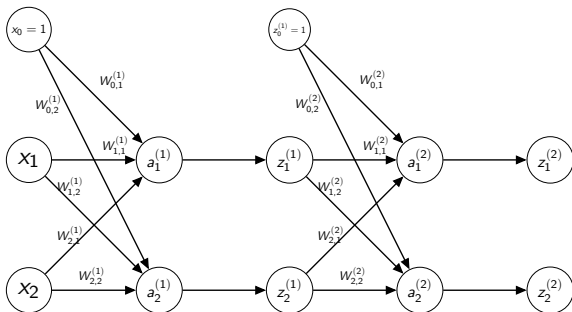
back-propagation implements stochastic gradient descent

Recall:

$$w_i \leftarrow w_i - \eta \frac{\partial \text{Error}(E, \vec{w})}{\partial w_i}$$

η : **learning rate**.

A 2-Layer Neural Network



Let $\hat{Y}$ be the output of a network (i.e. prediction).
For this network, $\hat{Y} = z^{(2)}$

The Backpropagation Algorithm

- An efficient method of calculating the gradients in a multi-layer neural network.
- Given training examples $(\vec{x}_n, \vec{y}_n)$ and an error/loss function $Error(\hat{Y}, Y)$. Perform 2 passes.
- **Forward pass:** compute the error $Error$ given the inputs and the weights.
- **Backward pass:** compute the gradients $\frac{\partial Error}{\partial W_{j,k}^{(2)}}$ and $\frac{\partial Error}{\partial W_{i,j}^{(1)}}$.
- Update each weight by the sum of the partial derivatives for all the training examples.

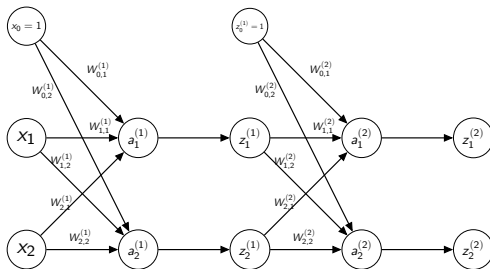
Forward Pass for a 2-layer Network

Calculate the values of $z_j^{(1)}$ and $z_k^{(2)}$ and *Error*.

$$a_j^{(1)} = \sum_i x_i W_{i,j}^{(1)} \qquad z_j^{(1)} = f(a_j^{(1)})$$

$$a_k^{(2)} = \sum_j z_j^{(1)} W_{j,k}^{(2)} \qquad z_k^{(2)} = f(a_k^{(2)})$$

$$\text{Error}(z^{(2)}, Y)$$

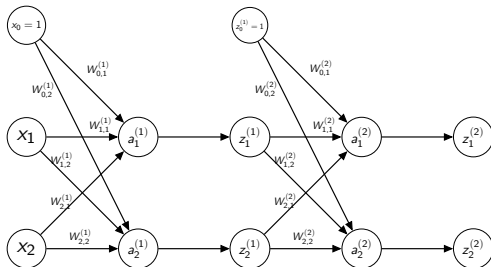


Backward Pass for a 2-layer Network

Calculate the gradients for $W_{i,j}^{(1)}$ and $W_{j,k}^{(2)}$.

$$\frac{\partial \text{Error}}{\partial W_{j,k}^{(2)}} = \frac{\partial \text{Error}}{\partial a_k^{(2)}} z_j^{(1)} = \delta_k^{(2)} z_j^{(1)}, \quad \delta_k^{(2)} = \frac{\partial \text{Error}}{\partial z_k^{(2)}} f'(a_k^{(2)})$$

$$\frac{\partial \text{Error}}{\partial W_{i,j}^{(1)}} = \frac{\partial \text{Error}}{\partial a_j^{(1)}} x_i = \delta_j^{(1)} x_i, \quad \delta_j^{(1)} = \left(\sum_k \delta_k^{(2)} W_{j,k}^{(2)} \right) f'(a_j^{(1)})$$



Some Calculus missing from the last slide

why does:

$$\delta_j^{(1)} = \frac{\partial \text{Error}}{\partial a_j^{(1)}} = \left(\sum_k \delta_k^{(2)} W_{j,k}^{(2)} \right) f'(a_j^{(1)})$$

$$\begin{aligned} \frac{\partial \text{Error}}{\partial a_j^{(1)}} &= \frac{\partial \text{Error}}{\partial z_j^{(1)}} \frac{\partial z_j^{(1)}}{\partial a_j^{(1)}} = \frac{\partial \text{Error}}{\partial z_j^{(1)}} f'(a_j^{(1)}) \\ &= \left(\sum_k \frac{\partial \text{Error}}{\partial z_k^{(2)}} \frac{\partial z_k^{(2)}}{\partial z_j^{(1)}} \right) f'(a_j^{(1)}) \\ &= \left(\sum_k \frac{\partial \text{Error}}{\partial z_k^{(2)}} f'(a_k^{(2)}) W_{j,k}^{(2)} \right) f'(a_j^{(1)}) \\ &= \left(\sum_k \delta_k^{(2)} W_{j,k}^{(2)} \right) f'(a_j^{(1)}) \end{aligned}$$

Example functions

Suppose our error/loss function is the mean squared error, our activation function is a sigmoid, and we have a batch of training examples, E , of size n . Then:

$$Error(e) = \frac{1}{n} \sum_{e \in E, k} (Y_k(e) - z_k^{(2)}(e))^2, \quad \frac{\partial Error}{\partial z_k^{(2)}} = -\frac{2}{n} \sum_{e \in E} (Y_k(e) - z_k^{(2)}(e))$$

$$f(a) = \frac{1}{1 + e^{-a}} \quad f'(a) = \frac{e^{-a}}{(1 + e^{-a})^2} = f(a)(1 - f(a))$$

With these, we can calculate $\delta_k^{(2)}$ for all k , then $\delta_j^{(1)}$ for all j .

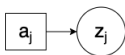
With the δ values, we can calculate all $\frac{\partial Error}{\partial W}$ for our weight updates.

The Recursive Relationship - Generalizing to More Layers

For unit j of layer ℓ , $\delta_j^{(\ell)} = \frac{\partial \text{Error}}{\partial a_j^{(\ell)}}$.

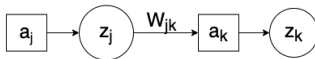
$$\delta_j^{(\ell)} = \begin{cases} \frac{\partial \text{Error}}{\partial z_j^{(\ell)}} \times f'(a_j^{(\ell)}), & \text{base case, } j \text{ is an output unit} \\ \left(\sum_k \delta_k^{(\ell+1)} w_{j,k}^{(\ell+1)} \right) \times f'(a_j^{(\ell)}), & \text{recursive case, } j \text{ is a hidden unit} \end{cases}$$

Base case:



Output layer

Recursive case:



Hidden Layer

Next layer

The Recursive Relationship - Generalizing to More Layers

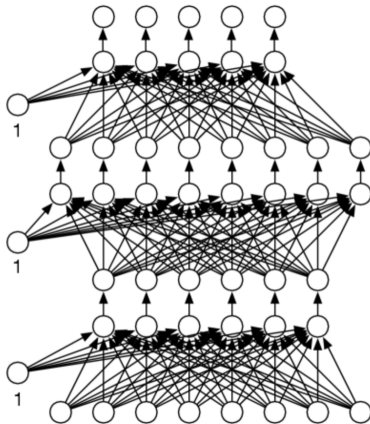
Then we can update our weights using:

$$W_{j,k}^{(\ell)} \leftarrow W_{j,k}^{(\ell)} - \eta \frac{\partial Error}{\partial W_{j,k}^{(\ell)}} \quad \text{where } \eta \text{ is the learning rate, and}$$

$$\frac{\partial Error}{\partial W_{j,k}^{(\ell)}} = \delta_k^{(\ell)} z_j^{(\ell-1)} \quad \text{for all layers beyond the input layer, or}$$

$$\frac{\partial Error}{\partial W_{j,k}^{(1)}} = \delta_k^{(1)} x_j \quad \text{at the input layer.}$$

Deep Neural Networks



output layer
activation function

complete linear function

hidden layer
activation function

complete linear function

hidden layer
activation function

complete linear function

input layer

Improving Optimization

- **Momentum**: weight changes accumulate over iterations
- **RMS-Prop**: rolling average of square of gradient
- **Adam**: combination of Momentum and RMS-Prop
- **Initialization**: randomly set parameters to start

Improving Generalization: Regularization

Regularized Neural nets: prevent **overfitting**, **increased bias** for **reduced variance**

- **parameter norm penalties** added to objective function
- **dataset augmentation**
- **early stopping**
- **dropout**
- **parameter tying**
 - **Convolutional** neural nets: used for images, parameters tied across space
 - **Recurrent** neural nets: used for sequences, parameters tied across time

Sequence Modeling

- **Word Embeddings**: latent vector spaces that represent the meaning of words in context
- **RNNs**: NN repeats over time and has inputs from previous time step
- **LSTM**: RNN with longer-term memory
- **Attention**: uses expected embeddings to focus updates on relevant parts of the network
- **Transformers**: multiple attention mechanisms
- **LLMs**: very large transformers for language (e.g. GPT-4)

Composite models and other learning methods

- **Random Forests**

- Each decision tree in the forest is different
- different features, splitting criteria, training sets
- average or majority vote determines output

- **Support Vector Machines**

- find the classification boundary with the **widest margin**
- combine with the **kernel trick**

- **Ensemble Learning**: combination of base-level algorithms

- **Boosting**

- sequence of learners fitting the examples the previous learner did not fit well
- learners progressively biased towards higher precision
- early learners: lots of false positives, but reject all the clear negatives
- later learners: problem is more difficult, but the set of examples is more focussed around the challenging boundary

Next

Unsupervised Machine Learning (Poole & Mackworth Chap. 10.3)